ENGINEER INTEGRATION GUIDE

LineFollowerPipeline

Pure-Java, zero-dependency line-following vision processor

Single-file library • Java 8+ • No external dependencies • Updated November 2026 (v3)

What's new in this version (v3 — path tracking)

- **The whole visible path is traced.** The "⊥" (scan strip + look-ahead) is replaced by a tracer that follows the line along its full visible length, on the floor (bird's-eye coordinates — only path points are converted, the image is never warped).
- **Local disturbances do not break the path.** Glare, reflections, shadows, uneven light, a colour cast, dirt, a patch of another colour or an object over the line: pixels that match neither learned colour are *unreliable* and the hidden section is bridged from the path memory and from the line before and after it — STEER and POWER stay as on the undisturbed path, for any colour pair.
- **Path memory.** The path is kept between frames and moved with the robot's estimated motion.
- **Pure-pursuit steering, smooth tracking.** No weaving, curves followed without cutting them; tracking from the sharpest curve ahead and the known path length, changing a few units per frame (no stop-and-go). The steering scale adapts to the robot.
- **Calibration learns the scene** — the working region (up to the table edge), the line width per row and the camera tilt, the straight-ahead line — and succeeds with a disturbance over part of the line.
- **No API break.** Only additions (Config, FrameData, constants). `pathTracking = false` runs v2 unchanged.

Changelog — What an Integrator Needs to Know NEW

In this guide NEW marks what v3 adds and CHANGED marks existing items whose behaviour changed in v3.

v3 — November 2026 (path tracking)

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
The whole visible path is traced — new `pathTracking` (on by default)	The "⊥" (scan strip + look-ahead column) is gone. Every frame the library follows the line along its **full visible length**, from just above the robot to where the floor stops being reliably visible, on the **floor** (bird's-eye coordinates — only the path points are converted, the image is never warped). See *How v3 Works*.	No
Local disturbances do not break the path — glare, reflections, shadows (the robot's own or an object's), uneven light, a colour cast, dirt, scratches, a patch of another colour, an object over the line	A pixel that matches **neither** the line nor the floor colour learned at calibration is **unreliable** — it counts as neither. A damaged section is bridged from the **path memory** and from the line before and after it, so **STEER and POWER stay the same** as on the undisturbed path. Works for any colour pair (dark on light, light on dark, coloured on coloured). See *Disturbances*.	No
Path memory — new `pathMemory`, `pathMemoryFrames`, `pathMaxGap`	The traced path is kept between frames on the floor and moved with the robot's motion (estimated from how the path shifts). A part that is hidden now — or already under the robot — is known from earlier frames.	No

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Pure-pursuit steering — new `pursuitLookAhead`, `pursuitPivotBehind`, `steerPerCurvature`, `steerAutoGain`	Steering aims at the point of the path about 1.2 camera heights ahead of the axle and is the curvature of the arc that reaches it: no weaving, curves followed without cutting them. **Range and sign unchanged** (−50 … +50, negative = turn left). The scale adapts itself to the robot (`steerAutoGain`).	No — see *Recommended Starting Values*
Smooth tracking — new `powerFullCurvature`, `powerCurvatureDeadband`, `powerPreview`, `powerRiseRate`, `powerFallRate`	Tracking (POWER) comes from the **sharpest curve ahead (and how near it is)** and from the **known length** of the path. It changes by at most a few units per frame — no more jumping between high values and the minimum (stop-and-go).	No
Calibration learns the scene — new `roiAuto`, `roiTopMarginFrac`, `learnCameraTilt`	Calibration finds the working region (ROI: from just above the robot to where the floor stops — table edge, background — minus a margin; full image width), the line width per row (perspective) and from it the **camera tilt**, the straight-ahead line and the heading zero. It succeeds even if glare or a shadow covers part of the path at that moment (those pixels are excluded).	No
New `FrameData` fields	The path points (with where each came from: measured now / memory / hidden-now / bridged), the look-ahead point, curvature, offset, heading, path length, confidence, share from memory, the ROI, a grid of unreliable cells, the estimated motion. See *FrameData Fields*.	No (optional for an overlay)
v2 still available	`cfg.pathTracking = false` runs the v2 pipeline unchanged (for comparison).	—

Do I need to change my integration code? No. No public method, constant or field was removed or renamed; v3 only adds Config fields, FrameData fields and constants. RoboPhoneApp and the Blockly block work unchanged. The **values** behave differently: steering is gentler on a pure sideways offset and follows curves exactly, tracking is smooth. Measured results: `_investigation/CHANGES_v3.md`.

v2 — October 2026

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Distractor rejection — new `rejectDistractors`, `bandMaxWidthMult`, `bandMinWidthMult` (on by default)	A dark object next to the track (a black chair leg, a bag, a shadow, the dark floor past a table edge) is no longer taken for the line. A band only counts as the line if its **width** and **line-ness** match the line measured at calibration. See *Distractor Rejection*.	No
Steering uses the whole ⊥ — new `steerOffsetWeight`, `steerHeadingWeight`, `steerLookAhead`, `steerHeadingMinRows`, `steerHeadingCalibMaxDeg`	Steering = **offset** of the line in the scan strip (as before) **+ heading** of the line from the look-ahead (new). A robot at an angle to the line now gets a steering value even when the line crosses the strip at the calibrated column (±15° → about ±8, ±30° → about ±17). Range and sign are unchanged. See *Steering from the Whole* ⊥.	No — but the steering values differ when the robot is at an angle, so re-check your steering gain

CHANGE	WHAT IT MEANS FOR YOU	CODE CHANGE NEEDED?
Hidden line is held — new `trackHoldFrames`	If the line disappears in the **middle** of the image (glare, a shadow, an object over it), the last steering and tracking are held for up to 6 frames instead of jumping to ±50 at once. During the hold `trackFound` is `false` and `lineHeld` is `true`. A line that leaves at the frame **edge** is still pinned to ±50 at once.	Only if your code reacts to `trackFound == false` on the very first frame
Line lock follows the predicted position	The lock gate is centred on where the line is expected (last position + recent motion) and can no longer grow to the whole frame. The lock correction (`trackRelockFrames`) needs 3× as many frames to leave a valid-looking line.	No
Calibration learns more	Besides the colours and the straight-ahead column, calibration now records the line **width**, how line-like its pixels are, and the **heading zero** (phone yaw on its mount). **Calibrate on a straight section, robot straight on the line.**	No
New `FrameData` fields	`steerOffsetTerm` , `steerHeadingTerm` , `headingDeg` , `headingConfidence` , `lineHeld` , `rejectedBands` — see *FrameData Fields*.	No (optional for a debug HUD)

Do I need to change my integration code? No. No public method, constant or field was removed or renamed — v2 only adds Config and FrameData fields, so existing code compiles and runs unchanged. What changes is the **behaviour**: steering now reacts to the robot's heading, distractors are ignored, and a line hidden mid-image is held briefly. To get the exact v1 behaviour back, set:

```
LineFollowerPipeline.Config v1 = new LineFollowerPipeline.Config();
v1.rejectDistractors  = false;   // original band selection (also turns off the hold)
v1.steerHeadingWeight = 0f;      // steering = offset only, exactly as in v1
pipeline.configure(v1);
```

Earlier changes

- **11 Oct 2026** — **Steering output is** `-50 … +50` (was `-100 … +100`): only the output is halved, the pipeline itself is unchanged. If your control program multiplies steering by a gain, **double the gain** to keep the same steering authority. `trackingMinValue` **default is** `5` (was `2`), so Tracking is `5 … 100` .
- **8 Oct 2026** — Automatic colour learning at calibration (`autoColor = true` by default; `trackIsBright` is now only the manual fallback); YUV 4:2:0 and ARGB input; Tracking ramps down with the remaining visible road; lock correction after turns (`trackRelockFrames`); `minContrast` ; richer `FrameData` (calibration phase / progress / problem, learned colours, threshold, line width, frame size).

What This Library Does

`LineFollowerPipeline.java` is a pure-Java, zero-dependency line-following vision processor. You feed it raw camera frames — **grayscale, YUV 4:2:0 or ARGB** — and it returns two integer control values your robot can act on directly: a **steering direction** and a **speed/tracking signal**. No OpenCV, no Android APIs, and no native libraries are required.

How v3 Works (path tracking) NEW

Everything below is on by default; `cfg.pathTracking = false` gives the v2 pipeline instead.

1. **Calibration learns the scene.** With the robot centred and straight on a straight section: the *colours* (pixels over a tall region above the robot, clustered; extra clusters are a second floor colour, the floor / line under other light, or a

disturbance — excluded from both models); the *working region (ROI)* — full image width, from just above the robot body to where the floor and line stop being reliably visible (table edge, background) minus a safety margin; the *geometry* — line width per row (rows with clean edges only) → camera tilt, line width on the floor, straight-ahead line and heading zero.

2. **Every frame the path is traced on the floor.** A chain of points one `pathStep` apart; at each point a short profile across the predicted path is sampled (~10 samples per line width) and every sample is *line*, *floor* or *unreliable*. A measurement must have the line colour, the expected width at that distance, clean edges (or one clean edge + the known width), and continue the path. The search starts from the remembered path; the whole width is searched only when the line is lost.
3. **Damaged sections are bridged, never guessed.** Unreliable points are filled from memory and from the measured path on both sides. Clean floor where the path should be is *absent* only if no more line is measured after it and the light is normal. Beyond the remembered path the tracer continues by curvature, bridges unreadable stretches (`pathMaxGap`), searches a sharp corner and looks past floor-coloured objects.
4. **Path memory.** The path is moved with the robot's motion (forward travel + rotation about the axle, matched to the measured path); points on the smooth continuation of the measured path are confirmed by it.
5. **Control.** A robust smooth curve through the path gives offset, heading, curvature and known length. *Steering*: pure pursuit to the point at `pursuitLookAhead` from the axle (shorter before sharp bends; raised if the path leaves the image side close in front). *Tracking*: the sharpest curve within `powerPreview` (relaxed with distance), the requested turn, the known length before a dead end, the confidence; rate-limited.

"Top to bottom" — both readings of the suggestion were measured in the simulation: working on the **floor** (bird's-eye view, path points only) is clearly better than in image coordinates, and tracing near → far with a second pass that re-reads gaps from both sides (`TRACE_BOTH`) is slightly better than one direction alone.

Disturbances — unreliable data instead of a wrong decision

A pixel is **line** only if it matches the learned line colour, **floor** only if it matches the learned floor colour (both relative to the local light level), and **unreliable** otherwise or if both match about equally well. The same rule works for every kind of disturbance and every colour pair:

DISTURBANCE	WHAT THE LIBRARY SEES	WHAT HAPPENS
Glare / reflection that whitens a dark line	unreliable pixels (off both colours); with a white floor the whitened tape looks like floor	the measured path before and after the spot plus the memory carry the hidden part; a stretch that looks like bare floor *before* more visible line is a gap, not an end
Glare on a white line / a bright stripe on a white-line track (looks like the line)	a line-coloured region that is **too wide** or **too thin** for the line at that distance	rejected by the expected width and the continuity; only an edge where the line's edge is predicted is used
Room-fixed shadow, uneven lighting	line and floor colours under a different light level	matched relative to the local light level
The robot's own shadow	moves with the robot: always in the same image place	the part under it was seen before it got there (memory) and continues the measured path
Patch of another colour under the line	floor = unreliable around the line	the line is still measured (one clean edge + the known width, or a full-width run)
Object covering the line / colour cast	unreliable (or floor-coloured) section	bridged from memory and continuity

Limits. If a disturbance hides *all* of the visible path for a long time (a large glare right in front of the robot while the path curves out of the image) the path is truly unknown: steering follows the remembered path and tracking decreases gradually. A black line whitened by glare on a *white floor* at the far end of the view cannot be told from the line's end until the line is seen beyond it.

Requirements

- Java 8 or higher
- One file only: `LineFollowerPipeline.java`
- No external dependencies

Minimal Integration (4 Steps)

1 Create one instance (keep it alive for the session)

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
```

2 Calibrate — feed frames until it returns `true`

```
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    grayscaleBytes = camera.getGrayscaleFrame();
}
```

3 Call `processFrame()` on every camera frame

```
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

4 Read the two output values

```
int steering = result[0];  // use for direction control
int tracking = result[1];  // use for speed control
```

Have colour frames? Use `calibrateYuv420(y, u, v, w, h)` / `processFrameYuv420(…)` (Android camera) or `calibrateArgb(…)` / `processFrameArgb(…)` instead — colour input separates even line/floor colours of the same brightness. See *Input Formats*.

Calibration is required. `processFrame()` must not be called until `calibrate()` has returned `true`. The robot must be correctly placed on the line — straight, on a straight section, with floor visible on both sides — while calibrating. See *Calibration* below.

Input Formats

Each format has a matching calibrate / process pair. Use the **same format** for both.

FORMAT	CALIBRATE	PROCESS	DATA
Grayscale	`calibrate(gray, w, h)`	`processFrame(gray, w, h)`	`byte[] gray` — `w × h` luma bytes
YUV 4:2:0 *(recommended on Android)*	`calibrateYuv420(y, u, v, w, h)`	`processFrameYuv420(y, u, v, w, h)`	`y` — `w × h` bytes; `u`, `v` — `((w+1)/2) × ((h+1)/2)` bytes each
ARGB	`calibrateArgb(argb, w, h)`	`processFrameArgb(argb, w, h)`	`int[] argb` — `w × h` packed `0xAARRGGBB`

Important: all arrays must be packed with **no row padding** (and the U/V planes with no pixel interleaving). Android CameraX planes are padded / interleaved — strip that before passing the data in (see `copyLumaPlane` / `copyChromaPlane` in RoboPhoneApp's `MainActivity`). Arrays that are too short throw `IllegalArgumentException`.

Which format should I use?

- **Colour (YUV / ARGB)** separates any two colours that look different — even two colours of the **same brightness** (e.g. red on green).
- **Grayscale** works for any two colours that differ in **brightness** (white on black, black on blue, white on brown, black on white). Calibration reports `ISSUE_LOW_CONTRAST` and keeps retrying if the two colours have the same brightness.

Output Format

`processFrame()` returns a **reused** `int[2]` array. Copy the values immediately if you need to keep them — the same array is overwritten on the next call.

INDEX	NAME	RANGE	MEANING
`result[0]`	**Steering**	-50 … +50	`0` = robot is on the line and parallel to it. **Negative = turn left** (the path ahead is to the left). **Positive = turn right**. v3: the curvature of the arc from the robot to the look-ahead point of the path (pure pursuit) — see *Steering — Pure Pursuit*. (v2: offset + heading of the ⊥.) `±50` toward the side where the line was lost (see *When the Line Is Lost or Hidden*).
`result[1]`	**Tracking**	5 … 100	`100` = straight path ahead, full speed. Drops before a curve (the sharper and the nearer, the lower) and ramps down as the known path ahead gets shorter (dead end). Changes smoothly (a few units per frame). Never falls below `Config.trackingMinValue` (default **5**), so the robot is never sent a full stop. Set that field to `0` for the full 0–100 range.

Typical robot wiring:

```
robot.setSteeringAngle(result[0]);   // direction
robot.setSpeed(result[1]);           // speed — naturally slows before turns
```

Optional Configuration

To tune the pipeline for your camera mount, track, or robot body size, create a `Config` object and pass it **before the first frame**:

```
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
// --- apply your settings here ---
pipeline.configure(cfg);  // must be called BEFORE the first processFrame()
```

Calling `configure()` later discards the calibration (colours given with `setTrackColors()` are kept), so calibrate again afterwards. The **Range** column gives the values the code accepts or that make sense; values outside it are clamped or have no useful effect.

v3 / v2. The v3 path tracker (default) uses *Line Colour*, *Camera Mount*, `trackLostGraceFrames`, `trackHoldFrames`, `trackLengthSlowdown`, `trackLengthFullFrac`, `trackingMinValue`, `calibFrames`, `steerHeadingCalibMaxDeg` and the five v3 tables at the end of this section. *Scan Strip*, *Look-Ahead Column*, *Track Memory*, *Distractor Rejection*, *Steering from the Whole* ⊥ and *Sensitivity* apply to the v2 pipeline (`pathTracking = false`). Distances in the v3 fields are in **camera heights** (1.0 = the camera's height above the floor).

Line Colour

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`autoColor`	boolean	true	true / false	**Normal setup.** Learn the line and floor colours during calibration — any two different colours work and `trackIsBright` is ignored. `false` = classic grayscale mode using `trackIsBright`.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`trackIsBright`	boolean	true	true / false	**Manual fallback only** — not needed with `autoColor = true`. Used when `autoColor = false`, and before the colours have been learned (e.g. if `processFrame()` is called before calibration has finished). `true` = bright line on dark floor. `false` = dark line on bright floor.
`colorCalibFrames` CHANGED	int	5	≥ 1	Frames sampled to learn the two colours.
`colorMinSeparation`	float	4.0f	≥ 0	Minimum line-vs-floor colour separation (in standard deviations of the colour noise). Below this, calibration discards the samples and retries.
`colorMinDistance`	float	12f	≥ 0	Minimum distance between the two learned colours (YUV grey-levels). Rejects "two colours" that are only sensor noise.
`colorMinThreshold`	int	112	0 … 255 (useful 64 … 192)	In colour mode the floor maps to 64 and the line to 192 internally; the threshold never drops below this so noisy floor pixels are never taken for the line.
`thresholdMargin`	int	0	grey-levels, ± (threshold kept in 15 … 240)	Bias added to the auto threshold. Increase if false positives occur. Decrease if the line is being missed.
`minContrast`	int	24	0 … 255	If the two halves of the Otsu split differ by less than this many grey-levels, the strip is a plain surface with **no line** (prevents "finding" a line in sensor noise). `0` disables.
`bottomIgnoreFrac` CHANGED	float	0.15f	0 … 0.9	Fraction of the frame bottom to ignore — use this to exclude the robot's own body from the image. Increase until the chassis disappears from the scan. (RoboPhoneApp uses `0.25`.)

Scan Strip (H-ROI — Near Line / Steering) — v2

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`hRoiCenterYFrac`	float	0.85f	0 … 1	Vertical position of the near scan strip as a fraction of frame height. `0.85` = 85% down from the top (near the bottom). The strip is moved up if it would overlap the ignored robot-body band.
`hRoiThickPx`	int	24	1 … frame height	Height of the scan strip in pixels.

Look-Ahead Column (V-ROI — Turn Preview / Tracking) — v2

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`vRoiHalfWidthPx`	int	80	≥ 1 px	Half-width in pixels of the look-ahead column. Increase on tight curves so the line stays inside the scan window. With `rejectDistractors` the window is automatically widened on rows where the line itself (plus a small margin) would not fit — e.g. wide tape at 960 × 1280.
`vRoiHeightFrac`	float	0.5f	0 … 1	How far ahead to look, as a fraction of frame height. `0.5` = half a frame ahead. Larger = more warning of far turns, but noisier.
`vRoiAdaptiveThreshold`	boolean	true	true / false	Gives the look-ahead its own Otsu threshold. **Recommended** — the far line is dimmer than the near line and benefits from separate calibration.

Camera Mount

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`cameraTiltDeg` **CHANGED**	float	45f	0 … 90 (degrees)	How far the phone's camera points below horizontal. Used for the perspective taper of the look-ahead, the look-ahead zero reference, the road-length slowdown, the line-width test of the look-ahead and the **heading** used by the steering (the line is projected onto the floor). Measure it on the real robot — a wrong value scales the heading part of the steering. Keep it large enough that the whole look-ahead lies below the horizon (floor only).
`verticalFovDeg`	float	60f	1 … 179 (degrees; phones ~55–65)	Vertical field of view of the camera (along the frame height). Most phone back cameras are ~55–65°. Used together with `cameraTiltDeg` for the floor projection.

Track Memory — v2

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`trackMemory`	boolean	true	true / false	Lock onto the line and only search a gate around its predicted position (last position + recent motion), so reflections elsewhere are ignored. `false` = search the whole width every frame.
`trackMemoryGateMult`	float	3.0f	> 0	Gate half-width as a multiple of the measured line width (with `rejectDistractors`, never more than an acceptable band width, so the gate cannot grow to the whole frame). The gate is never wider than half the frame.
`trackMemoryMinGatePx`	int	48	≥ 0 px	Minimum gate width in pixels regardless of line width.
`trackLostGraceFrames` **CHANGED**	int	8	≥ 0 frames	Frames to wait after losing the line before searching the full frame for a new one. At 30 fps, `8` ≈ 0.25 seconds.
`trackRelockFrames`	int	6	≥ 0 frames	Lock correction. If a band that looks much more like the line (near the calibrated straight-ahead column, with the calibrated line width) is seen outside the lock for this many frames in a row, the lock moves to it — so a reflection or neighbouring tape grabbed after a turn cannot keep steering offset. With `rejectDistractors`, a locked line that still looks valid is only abandoned after **3×** this many frames. `0` disables.

When searching for the line (start-up, or after it was lost), the pipeline no longer simply takes the biggest band in view: it prefers the band nearest the calibrated straight-ahead column whose width matches the line width measured during calibration. While the locked line is lost at the frame edge, steering reads ±50 toward the side it left; a line that vanishes mid-image is held first (see *When the Line Is Lost or Hidden*).

Distractor Rejection (chair, bag, shadow, table edge) — v2

Objects with a colour close to the line's (a black chair leg on a black-tape track, a shadow on wood, the dark floor beyond a table edge) pass the threshold exactly like the line. With `rejectDistractors` on, a band only counts as the line if it **looks like the line learned at calibration**:

- **Width** — the band must be between `bandMinWidthMult` × and `bandMaxWidthMult` × the calibrated line width (the maximum is corrected for the line's slant in the image). Wide blobs, dark regions at the frame edge and thin specks are rejected.
- **Line-ness** — among acceptable bands, the one whose pixels are as line-like as the calibrated line wins (a shadow on wood only just passes the threshold; the tape is well past it).
- **Continuity** — while locked, the band nearest the *predicted* position wins (on the scale of one line width), and a band that is the line **merged** with a blob or shadow is cut back to the part that continues the line (its strong core, or

the edge that sits where the line's edge was).

- **Look-ahead** — the row-by-row tracker of the V-ROI applies the same width test on every row (perspective-scaled), so it cannot step onto a nearby object; a run cut by the scan window is only used through the edge that continues the line.
- **Hidden line** — if the locked line disappears in the middle of the image (glare, a shadow or an object over it) the last steering and tracking are held for `trackHoldFrames`; a line that leaves at the frame edge (a real turn) is still pinned to that edge at once.
- **Relock** — the lock correction (`trackRelockFrames`) still exists, but a valid-looking locked line is only abandoned after three times as many frames, so a chair leg near the centre cannot steal the lock during a turn.

What you must ensure — the rules compare everything with the line measured at calibration, so:

- **Calibrate with the robot on the line**, straight, on a straight section, with floor visible on both sides, and **no distractor inside the scan strip** during calibration.
- **Use one tape width for the whole track** (a reasonable width: the line should cover well under half of the scan strip). Width changes of more than about 2× (`bandMaxWidthMult`) or less than 0.3× (`bandMinWidthMult`) are treated as "not the line". **Recalibrate** when you change tape.
- Set `cameraTiltDeg` / `verticalFovDeg` close to the real camera — the look-ahead width test uses them.
- Remaining limit: a dark object of **exactly the tape's width**, lying where the line is predicted to be, can still be taken for the line.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`rejectDistractors`	boolean	true	true / false	Turn the rules above on. `false` = the original (v1) band selection (this also turns off the hold and the predicted gate).
`bandMaxWidthMult`	float	2.0f	≥ 1.3 (useful 1.5 … 3)	Widest band accepted as the line, as a multiple of the expected (calibrated, slant-corrected) width. A real line stays within about ±30 %; a line merged with a blob is ≥ 2.5×.
`bandMinWidthMult`	float	0.3f	0 … 1 (0 = no minimum)	Narrowest band accepted (fraction of the calibrated width); bands cut by the frame edge are exempt.
`trackHoldFrames` **CHANGED**	int	6	0 … trackLostGraceFrames	Frames the last output is held when the line vanishes mid-image (6 ≈ 0.1 s at 60 Hz). `0` = report the loss at once (v1). During the hold `trackFound` is `false` and `FrameData.lineHeld` is `true`. Only active with `rejectDistractors = true`.

Steering from the Whole ⊥ (offset + heading) — v2

The pipeline looks at the line as a ⊥ (an inverted "T"): the **base** is the horizontal scan strip (H-ROI), the **vertical part** is the line followed up the look-ahead (V-ROI). The v1 steering used only the base of the ⊥ — where the line crosses the scan strip — so a robot at 30° to the line and a robot straight on it read the **same** value if the line crossed the strip at the same point. The steering now also uses the line's **heading** from the vertical part of the ⊥:

```
offset   = (centroid − calibrated column) / (W/2) × 100                    (v1 steering)
heading  = 100 × L × tan(ψ − ψ₀) / (floor half-width seen by the H-ROI)    (ψ − ψ₀ limited to ±75°)
L        = steerLookAhead × (floor distance from the camera to the H-ROI)
steering (internal) = clamp(steerOffsetWeight × offset + steerHeadingWeight × c × heading, −100, +100)
result[0] = round(steering / 2)                                            → [-50 … +50]
```

- **W** is the frame width in pixels; the **centroid** is where the line crosses the scan strip; the **calibrated column** is the straight-ahead column measured during calibration (`FrameData.steerRefXFrac`).

- **ψ** is the direction of the line on the floor where it crosses the scan strip: the H-ROI centroid and the look-ahead rows on the line are projected onto the floor with the camera geometry (`cameraTiltDeg`, `verticalFovDeg`, exact pinhole model) and fitted with a parabola; its tangent at the strip is ψ. Positive = the line heads to the right.
- **ψ_0** is the heading measured during calibration (phone yaw on its mount), so a robot correctly placed on a straight line still reads `0`. Calibrate on a straight section; a calibration heading larger than `steerHeadingCalibMaxDeg` is ignored ($\psi_0 = 0$).
- Together the two parts are the lateral position of a **pure-pursuit target point** a distance L further along the line, in the same units as the v1 steering.
- **c** (the heading confidence, `FrameData.headingConfidence`, 0 … 1) fades the heading out when too little of the line is visible ahead (end of line, line leaving the frame) or when the line is not a smooth curve (a 90° corner, a junction) — the steering then falls back to the v1 offset only.
- **Signs**: both parts are positive when the robot must turn **right** (line to the right / line heading right). `FrameData.steerOffsetTerm` and `steerHeadingTerm` are the two weighted parts in output units (already halved), so before clamping and rounding `steering ≈ steerOffsetTerm + steerHeadingTerm`.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`steerOffsetWeight`	float	1.0f	≥ 0 (keep 1)	Weight of the offset (v1) part.
`steerHeadingWeight`	float	1.0f	≥ 0	Weight of the heading part. `0` **= exactly the v1 steering.**
`steerLookAhead`	float	0.5f	> 0 (tested 0.25 … 1.0)	L as a multiple of the camera-to-strip floor distance. Larger = the heading counts more: a robot at an angle to the line straightens out sooner, but it runs a little more on the inside of tight curves and, with a high steering gain plus delay, can start to weave. `0.5` was the best balance in the closed-loop simulation. With the defaults, a robot at ±15° whose line crosses the strip at the calibrated column reads about ±8, at ±30° about ±17.
`steerHeadingMinRows`	int	24	≥ 1 rows	Look-ahead rows on the line needed for the heading at full weight.
`steerHeadingCalibMaxDeg`	float	10f	≥ 0 (degrees)	Largest heading accepted as the calibration zero.

Sensitivity & Tracking — v2 (tracking fields also v3)

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`minWeightFrac`	float	0.01f	0 … 1	Minimum fraction of scan strip pixels that must be "track" to trust the detection. Raise to reject weak signals.
`trackOffsetFullFrac`	float	1.0f	> 0 (usually 0.5 … 1)	How far the look-ahead line must lean (as a fraction of half-frame width) to push Tracking to its minimum. Lower (e.g. `0.6`) makes turns read earlier.
`trackLengthSlowdown` CHANGED	boolean	true	true / false	Also limit Tracking by how much road is still visible ahead (floor distance, perspective-corrected). Approaching a dead end or a sharp corner, Tracking ramps down **linearly with the remaining road** instead of dropping from 100 to the minimum in one frame. A fully visible straight road still reads 100.
`trackLengthFullFrac` CHANGED	float	0.8f	0 … 1	Fraction of the look-ahead distance the line must reach for the road-length part to read 100. Keeps the dim far edge from lowering Tracking on straight roads.
`trackingMinValue`	int	5	0 … 100	Floor for the Tracking output. `result[1]` will never go below this value, so the robot never receives a full stop signal. Set to `0` to allow the full 0–100 range.

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`curveFullScaleDeg`	float	30f	—	**Not used** by the current code (no effect). Kept so existing code that sets it still compiles.

Tracking Minimum Value

By default Tracking **never reaches zero**: `result[1]` is clamped to a floor of 5, so the robot always receives at least a small speed signal. This prevents mechanical problems caused by a complete stop command.

VALUE	EFFECT
5 *(default)*	Tracking minimum is 5 — the robot always gets at least a small speed signal.
10	Raise the floor further.
0	Disable the floor — allow the full 0–100 range.

```
cfg.trackingMinValue = 5;   // set before calling pipeline.configure(cfg)
```

Calibration Frames

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`calibFrames` CHANGED	int	10	≥ 1	Number of solid detections `calibrate()` collects before locking the straight-ahead zero reference (it takes their median), the line width, the line-ness level and the heading zero. Assumes the robot is correctly placed on the line. Increase for more robust calibration.

Path Tracking (v3) — Working Region and Geometry NEW

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`pathTracking` NEW	boolean	true	true / false	**v3** path tracking. `false` = the v2 ⊥ pipeline, unchanged (for comparison). Before calibration frames are processed by the v2 pipeline.
`roiAuto` NEW	boolean	true	true / false	Find the top of the working region during calibration (where the floor / line stop being reliably visible). `false` = up to the top of the image. The bottom is always just above the robot body (`bottomIgnoreFrac`).
`roiTopMarginFrac` NEW	float	0.10f	0 … 0.5	Safety margin below the found top, as a fraction of the visible floor distance.
`learnCameraTilt` NEW	boolean	true	true / false	Learn the camera tilt from how the line width changes with the row during calibration. Used only when consistent and within 25° of `cameraTiltDeg` ; otherwise `cameraTiltDeg` is used. `FrameData.cameraTiltUsedDeg` shows the value used.

Path Tracing and Memory (v3) NEW

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`pathStep` NEW	float	0.10f	0.02 … 0.3	Distance between two path points (camera heights).
`pathTraceDirection` NEW	int	TRACE_BOTH	TRACE_BOTTOM_UP, TRACE_TOP_DOWN, TRACE_BOTH	Order of tracing: near → far, far → near, or near → far plus a second pass that re-reads the gaps between two good sections from both sides (measured best).

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`pathFloorSpace` NEW	boolean	true	true / false	Trace, fit and remember the path on the floor (bird's-eye; only path points are converted). `false` = the same algorithm in image coordinates (comparison only — clearly worse).
`pathMemory` NEW	boolean	true	true / false	Keep the path between frames, moved with the robot's motion. `false` = every frame from scratch (comparison only).
`pathMemoryFrames` NEW	int	120	≥ 1 frames	How long a remembered point stays trusted without being confirmed (its confidence halves after ~0.7 × this while nothing is measured, ~4× longer while the path is tracked). Also the longest time the robot follows a remembered path while it is hidden or out of view in front of the robot.
`pathMaxGap` NEW	float	2.5f	≥ 0	Longest unreadable stretch (camera heights) bridged by continuity where there is no memory of the path.

Disturbance Handling (v3) NEW

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`colorMatchSigma` NEW	float	4.0f	2 … 8	Colour match tolerance in standard deviations of the colours learned at calibration. A pixel within this of neither colour is unreliable. Smaller = more pixels unreliable; larger = disturbances may pass as line or floor.
`lightLocalRange` NEW	float	1.6f	1.05 … 3	Brightness change (×/÷) allowed around the local light level (measured from the floor next to the line) when matching colours. With brightness-only input (grayscale, `autoColor = false`) at most 1.3 is used.

Steering — Pure Pursuit (v3) NEW

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`pursuitLookAhead` NEW	float	1.2f	0.5 … 3	Distance (camera heights) from the robot's turning point to the target point on the path. Larger = smoother, more inclined to cut tight curves; smaller = tighter, more weaving. Shortened automatically before sharp bends (never below 0.5).
`pursuitPivotBehind` NEW	float	0.4f	0 … 2	The robot's turning point (axle centre) behind the camera's floor point, camera heights (0.4 = 8 cm at 20 cm height).
`steerPerCurvature` NEW	float	30f	> 0	Steering per unit of pursuit curvature: steering = this × 2·lateral / distance^2 of the target point (limited to ±50). Suits "move steering" with gain 1 on a robot whose wheelbase is about 0.65 camera heights.
`steerAutoGain` NEW	boolean	true	true / false	Scale the steering to the robot automatically (×0.5 … ×2.5): while the robot follows a curve steadily it turns exactly as much as the path curves, so the ratio of path curvature to requested curvature reveals the program's gain and the robot's response.

Tracking from the Path (v3) NEW

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`powerFullCurvature` NEW	float	1.5f	> powerCurvatureDeadband	Curvature (1 / camera height) at which tracking reaches its minimum (1.5 = radius 0.67 camera heights, 13 cm at 20 cm height).

FIELD	TYPE	DEFAULT	RANGE	DESCRIPTION
`powerCurvatureDeadband` NEW	float	0.2f	≥ 0	Curvature up to which tracking stays 100 (gentle curves at full speed).
`powerPreview` NEW	float	2.0f	> 0	How far ahead (camera heights) curves count; a curve's effect fades with its distance.
`powerRiseRate` NEW	float	3f	> 0	Largest rise of the tracking output per frame.
`powerFallRate` NEW	float	8f	> 0	Largest fall per frame (a truly lost line still drops at once).

`trackLengthSlowdown` / `trackLengthFullFrac` keep their meaning: tracking ramps down when the known path ends on clean floor before `trackLengthFullFrac` of the view's depth.

Calibration

Before calling `processFrame()` you must calibrate. Place the robot correctly on the line, then call `calibrate()` (or the YUV / ARGB version) once per camera frame until it returns `true`:

```
// Phase 1 — Calibration (at startup, or when the user presses calibrate)
while (!pipeline.calibrate(grayscaleBytes, width, height)) {
    // keep feeding frames until calibration is complete
    grayscaleBytes = camera.getGrayscaleFrame();
}

// Phase 2 — Normal operation
int[] result = pipeline.processFrame(grayscaleBytes, width, height);
```

Calibration has two phases:

1. **Learn the colours** (`colorCalibFrames`, default 5 frames). The scan strip is sampled and split into two colour clusters (k-means). The **floor** is the cluster that fills both the left and right frame edges; the other is the **line**. If that fails — typically a shadow or uneven light making part of the floor almost as dark as a dark line — 3 and then 4 clusters are tried: the line is the cluster that forms one band and never reaches the frame edges, and all other clusters (lit floor, shadowed floor…) count as floor. A Fisher linear discriminant then gives the colour direction that best separates line from floor, and every later frame is converted to a "line-ness" image (line bright, floor dark) before the usual Otsu + line-lock processing. Skipped when `autoColor = false` or when the colours were given with `setTrackColors()`.
2. **Find straight-ahead** CHANGED Collects `calibFrames` (default 10) solid line detections — preferring the band where colour learning saw the line and, with `rejectDistractors`, of the width colour learning measured — and takes their **median** as: the steering zero reference (straight-ahead column); the typical **line width** and **line-ness level** (used by distractor rejection); and the **heading zero** ψ_0 (only when `steerHeadingWeight ≠ 0`): the median line heading of the frames that saw enough look-ahead. It is used only if at least half of the frames saw it and it is within `steerHeadingCalibMaxDeg`; otherwise $\psi_0 = 0$.

v3 (default) CHANGED Phase 1 samples a tall region above the robot instead of the strip and keeps further colour clusters apart as disturbances; phase 2 scans the rows from the robot to the top of the image in each of the `calibFrames` frames and derives the working region (ROI), the line width per row and the camera tilt, the line width on the floor, the straight-ahead line and the heading zero, and the initial path. Rows hidden by a disturbance are not used, so calibration also works with glare or a shadow over part of the line.

With the defaults calibration needs at least **15 frames** (5 + 10). It returns `true` once both phases are done. Progress and problems are reported in the callback (`calibPhase`, `calibProgress`, `calibIssue`):

CALIBISSUE	MEANING	WHAT TO DO
`ISSUE_NONE`	All good	—
`ISSUE_LOW_CONTRAST`	The two colours in view are too similar, or there is only one surface	Make sure the line is in the STEER strip; for same-brightness colours use YUV/ARGB input; even out the lighting
`ISSUE_NO_FLOOR`	No colour fills both frame edges	Place the robot **on** the line, with floor visible on both sides
`ISSUE_NO_LINE`	Colours learned, but no line band found this frame	Check the line is inside the STEER strip

Before you calibrate — checklist

- The robot stands **on the line, straight and centred**, on a **straight section** (the heading zero and the straight-ahead line are learned here).
- **Floor visible on both sides** of the line (both image edges must show floor) in the lower part of the image.
- v3: a glare spot or a shadow over part of the line is tolerated; a second tape or a large object of the line's colour near the line should be avoided.

- The tape width is the one used on the whole track.

If calibration cannot complete, `calibrate()` simply keeps returning `false` — there is no timeout. Show `calibIssue` to the user, fix the cause (see the table) and keep feeding frames; call `requestRecalibration()` to start again from phase 1. Do **not** drive on `processFrame()` output before calibration is done: until then the steering zero is the image centre, there is no calibrated line width (so distractor rejection cannot work), and if the colours were not learned yet the frame is read as plain brightness with `trackIsBright` polarity.

Giving the colours yourself (optional)

```
pipeline.setTrackColors(0xFF000000 /* black line */, 0xFF1E3CC8 /* blue floor */);
pipeline.clearTrackColors();   // go back to learning them
```

`setTrackColors()` has no effect when `autoColor = false`, and throws `IllegalArgumentException` if the two colours are the same.

Re-calibrating mid-run

```
pipeline.requestRecalibration();
// then feed frames to calibrate() again before resuming processFrame()
```

Recalibrate whenever you move to a track with different colours, very different lighting or a different tape width.

When the Line Is Lost or Hidden NEW

SITUATION	STEERING RESULT[0]	TRACKING RESULT[1]	TRACKFOUND	LINEHELD
Line visible	offset + heading	from the look-ahead	`true`	`false`
Line disappears in the **middle** of the image (glare, shadow, object over it) — first `trackHoldFrames` frames (default 6)	**last value held**	**last value held**	`false`	`true`
Still hidden after the hold	±50 toward the side the line was last on	trackingMinValue (5)	`false`	`false`
Line leaves at the frame **edge** (sharp turn, robot drove off)	±50 toward that edge, at once	trackingMinValue (5)	`false`	`false`
Line comes back inside the lock gate (where it left) within `trackLostGraceFrames` (default 8)	normal again — the **same** line is re-locked	normal	`true`	`false`
Gone for more than `trackLostGraceFrames`	stays at the last edge value (±50, slightly less if the calibrated column is far off-centre)	trackingMinValue	`false`	`false`
A new line appears after that	full-width search: the band nearest the calibrated column with the calibrated width is taken	normal	`true`	`false`

Lost or hidden line with path tracking (v3) NEW

SITUATION (V3)	STEERING	TRACKING	TRACKFOUND	LINEHELD
Path measured this frame (also when parts are hidden — they come from memory)	pure pursuit on the path	from the path	`true`	`false`
Nothing measured, the remembered path lies in **unreliable** pixels (glare, shadow, object over all of the visible line)	pure pursuit on the remembered path	from it, decreasing gradually with the confidence	`false`	`true`
Nothing measured, the remembered path has moved **below the view** (a sharp corner just in front of the robot)	pure pursuit on the remembered path, up to `pathMemoryFrames`	from it	`false`	`true`
Clean floor where the line should be, in the middle of the image	last value held for `trackHoldFrames` (tracking keeps ramping down if the line's end was seen)	held	`false`	`true`
Line left through the image **side**, or still gone after the above	`±50` toward the side the robot was turning to / the path leads	`trackingMinValue`	`false`	`false`
Gone for more than `trackLostGraceFrames`	the whole view is searched every frame; a line found (nearest straight ahead / the side it left) replaces the memory	normal once found	—	—

Use `trackFound` (not `tracking == 5`) to show "TRACK LOST" — a hard turn also reads 5. With `trackMemory = false` there is no lock, no pinning and no hold: a missing line keeps the steering of its last position and tracking drops to the minimum. If calibration has not completed, see *If calibration cannot complete* above.

Runtime Utility Methods

```
int     fps       = pipeline.getFramesPerSecond();  // frames in the last completed second
long    frames    = pipeline.getTotalFrames();      // cumulative frame count since init
double  seconds   = pipeline.getElapsedSeconds();   // wall-clock seconds since first frame
boolean ready     = pipeline.isCalibrated();        // true once calibration is complete
boolean colours   = pipeline.isColorModelReady();   // true once colours are learned (or preset)
int     lineArgb  = pipeline.getLineColorArgb();    // learned line colour, 0xAARRGGBB
int     floorArgb = pipeline.getFloorColorArgb();   // learned floor colour, 0xAARRGGBB
boolean init      = pipeline.isInitialized();       // false before 1st frame / after release()
pipeline.requestRecalibration();  // re-learn colours + straight-ahead; then calibrate() again
pipeline.release();               // reset; the next frame re-initialises (calibrate again)
pipeline.init(960, 1280, cfg);    // optional: initialise now (size + config; drops calibration)
pipeline.setFrameCallback(cb);    // register overlay callback; pass null to unregister
```

Geometry getters (the same values the frame callback delivers — prefer the callback):

```
int     imgW    = pipeline.getImgW();                // frame size in pixels
int     imgH    = pipeline.getImgH();
int     stripTop = pipeline.getHRoiTop();            // scan-strip rows [top, bottom)
int     stripBot = pipeline.getHRoiBot();
int     vTop    = pipeline.getVRoiTop();             // far edge row of the look-ahead
int     vHalf   = pipeline.getVRoiHalfWidthPx();     // look-ahead half-width (config)
float   steerRef = pipeline.getSteerRefXFrac();      // calibrated straight-ahead columns
float   trackRef = pipeline.getTrackRefXFrac();
int     nPts    = pipeline.getLookAheadPointCount();  // dynamic road path (shared buffers)
float[] cols    = pipeline.getLookAheadColFracs();
float[] rows    = pipeline.getLookAheadRowFracs();
float   horizon = pipeline.getHorizonYFrac();        // floor horizon row from the mount tilt
float   scale   = pipeline.perspectiveWidthScale(0.5f); // relative apparent width at a row
```

Frame Callback — Overlay Geometry

Register a single callback once, and the library calls it automatically on **every calibration frame and every `processFrame()` frame**, passing a `FrameData` object with everything needed to draw a visual overlay and a status display: in v3 the working region, the traced path with the source of every point, the look-ahead target and the unreliable cells; in v2 the H-ROI strip, the V-ROI look-ahead band and the live centroid; calibration progress and the learned colours in both.

No extra computation is added. The data already exists inside the pipeline — the callback just packages and delivers it (≈ 0.02 ms per frame). The callback runs on the thread that called calibrate / processFrame.

```
pipeline.setFrameCallback(data -> {
    // called automatically every frame
    // use data fields to draw your overlay
});

// To unregister:
pipeline.setFrameCallback(null);
```

Important: the same `FrameData` instance is reused on every call. Copy any fields you need to keep beyond the current frame.

FrameData Fields

FIELD	TYPE	DESCRIPTION
`imgW`, `imgH`	int	Frame size in pixels — the image all fractions refer to.
`hRoiTopFrac`	float	H-ROI top edge — fraction of frame height.
`hRoiBotFrac`	float	H-ROI bottom edge — fraction of frame height.
`vRoiTopFrac`	float	V-ROI far (top) edge — fraction of frame height.
`steerRefXFrac`	float	Calibrated straight-ahead column for steering — fraction of frame width.
`trackRefXFrac`	float	Calibrated straight-ahead column for look-ahead — fraction of frame width.
`halfBotFrac`	float	Near (bottom) half-width of the V-ROI box — fraction of frame width.
`halfTopFrac`	float	Far (top) half-width of the V-ROI box — fraction of frame width. Smaller than `halfBotFrac` due to perspective taper.
`ignoreTopFrac`	float	Top of the ignored robot-body band — fraction of frame height (`1` = nothing ignored).
`centroidXFrac` CHANGED	float	Live detected line position — fraction of frame width (the real position, also during calibration). `-1` if the line was not found (also while it is held).
`trackWidthFrac`	float	Measured line width — fraction of frame width.
`tiltDeg`	float	Lean of the look-ahead line from vertical **in the image**, degrees (positive = leans right). For drawing only — the steering uses `headingDeg`. `0` before calibration or when the line is not found.
`lookAheadColFrac[]` CHANGED	float[]	Column positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadRowFrac[]`	float[]	Row positions of the dynamic road poly-line, nearest to farthest. Array size **48**.
`lookAheadPointCount`	int	Number of valid points in the path arrays. Only read up to this index.
`trackFound`	boolean	`true` when the line was detected in the H-ROI this frame. Use this for "TRACK LOST" — not `tracking > 5` (a hard turn also reads 5).
`isCalibrated`	boolean	`true` once calibration is complete.
`calibPhase`	int	`CALIB_LEARNING_COLORS`, `CALIB_FINDING_CENTER` or `CALIB_DONE`.
`calibProgress`	float	Overall calibration progress, `0 … 1`.
`calibIssue`	int	Current calibration problem (`ISSUE_*`, see *Calibration*).
`colorModelReady`	boolean	`true` when frames are processed in learned-colour mode.
`lineColorArgb`, `floorColorArgb`	int	Learned colours (`0xAARRGGBB`) — handy for UI swatches. `0` until learned.
`colorSeparation`	float	How well the two colours separate, in noise standard deviations.
`threshold`	int	Threshold used in the scan strip this frame; `-1` = no contrast (no line).
`steering` CHANGED	int	Steering value, `-50 … +50` — same as `result[0]`. `0` during calibration.
`tracking` CHANGED	int	Tracking value, `min … 100` — same as `result[1]`. `100` during calibration.
`steerOffsetTerm`	float	Offset (v1) part of the steering, in steering output units (`steerOffsetWeight` applied). `0` during calibration and when the line is not found.

FIELD	TYPE	DESCRIPTION
`steerHeadingTerm`	float	Heading part of the steering, in steering output units (already multiplied by `steerHeadingWeight` and `headingConfidence`). Before clamping and rounding, `steering ≈ steerOffsetTerm + steerHeadingTerm`. `0` during calibration and when the line is not found.
`headingDeg`	float	Line heading relative to the robot on the floor, perspective-corrected, minus the calibrated zero. Degrees, positive = the line heads to the right. `0` when unknown (line not found, too little look-ahead, or `steerHeadingWeight = 0`).
`headingConfidence`	float	Weight `0 … 1` the heading got this frame (`0` = too little / not a smooth line ahead → offset-only steering; also `0` when `steerHeadingWeight = 0`).
`lineHeld`	boolean	`true` while the line is momentarily hidden mid-image and the last output is held (`trackHoldFrames`).
`rejectedBands` CHANGED	int	Number of candidate bands in the scan strip rejected or trimmed as distractors this frame (`0` with `rejectDistractors = false` and during calibration). v3: number of path measurements trimmed or rejected as too wide / too thin.
`pathMode` NEW	boolean	`true` when the frame was processed by the v3 path tracker.
`roiTopFrac`, `roiBotFrac` NEW	float	The working region (full width) — fractions of frame height.
`pathColFrac[]`, `pathRowFrac[]` NEW	float[]	The path, nearest to farthest, in image fractions (may lie outside `0 … 1` for remembered points out of view). Array size `PATH_MAX_POINTS` (**64**). During calibration: the rows where the line was found.
`pathFwd[]`, `pathLat[]` NEW	float[]	The same points on the floor, robot frame, camera heights (forward / lateral, + = right).
`pathSource[]` NEW	byte[]	Where each point came from: `PATH_SRC_FRESH` (measured now), `PATH_SRC_MEMORY` (remembered: out of the ROI or not readable), `PATH_SRC_UNRELIABLE` (in an unreliable region now, filled from memory), `PATH_SRC_PREDICTED` (bridged by continuity).
`pathPointCount` NEW	int	Number of valid path points.
`pathFreshPoints`, `pathMemoryPoints`, `pathUnreliablePoints` NEW	int	Counts by source (whole path).
`pathMemoryFrac` NEW	float	Fraction `0 … 1` of the path ahead (within `powerPreview`) that came from memory.
`lookAheadValid`, `lookAheadXFrac`, `lookAheadYFrac`, `lookAheadFwd`, `lookAheadLat` NEW		The pure-pursuit target point: image fractions and floor (robot frame, camera heights).
`pathCurvature`, `pathCurvatureAhead` NEW	float	Curvature at the robot and the sharpest curvature ahead (1 / camera height, + = curves right).
`pathOffset`, `pathHeadingDeg` NEW	float	Lateral offset (camera heights, + = path right of the robot) and direction (degrees, + = heads right) of the path at the camera's floor point.
`pathLength` NEW	float	Length of the known path ahead (camera heights).
`pathConfidence` NEW	float	`0 … 1`: how well the path ahead is known (1 = measured now, or hidden but seen on both sides; drops only while it is truly unknown).

FIELD	TYPE	DESCRIPTION
`unreliableGrid` NEW	byte[]	`RELIABILITY_GRID_COLS × RELIABILITY_GRID_ROWS` (24 × 32) cells over the ROI, row-major from its top-left; `1` = pixels match neither colour (glare, shadow edge, object, colour patch).
`unreliableCells` NEW	int	Number of unreliable cells.
`motionFwd`, `motionLat`, `motionYawDeg` NEW	float	Estimated robot motion per frame (camera heights, degrees + = right).
`cameraTiltUsedDeg`, `lineWidthFloor` NEW	float	Camera tilt used for the floor geometry; line width on the floor (camera heights).

Drawing the overlay

Every field is a **fraction**, so convert to screen pixels first:

```
float screenX = data.centroidXFrac * screenWidth;   // any …XFrac field
float screenY = data.hRoiTopFrac   * screenHeight;  // any …YFrac / …TopFrac / …BotFrac field
```

Cropped previews: if the preview is scaled to fill the screen (Android `PreviewView` default FILL_CENTER), part of the image is cropped. Map with the same scale, or the overlay will drift: `scale = max(viewW / imgW, viewH / imgH)`, `x = (viewW − imgW·scale)/2 + xFrac·imgW·scale` (same for y). RoboPhoneApp's `OverlayView` does this.

v3 overlay NEW: draw the ROI (`roiTopFrac` … `roiBotFrac`, full width), the cells of `unreliableGrid` (24 × 32 over the ROI), the path `pathColFrac[i]` / `pathRowFrac[i]` for `i < pathPointCount` coloured by `pathSource[i]` (measured now / memory / hidden-now / bridged), and the target `lookAheadXFrac` / `lookAheadYFrac` when `lookAheadValid`. RoboPhoneApp's `OverlayView` is a complete example. The v2 drawing below applies when `pathMode` is false.

H-ROI strip (v2 — a full-width horizontal band):

```
float top    = data.hRoiTopFrac * screenHeight;
float bottom = data.hRoiBotFrac * screenHeight;
canvas.drawRect(0, top, screenWidth, bottom, paint);
```

V-ROI trapezoid (perspective-tapered box — the far edge is narrower, and leans toward `trackRefX`):

```
float centre = data.steerRefXFrac * screenWidth;
float shear  = (data.trackRefXFrac - data.steerRefXFrac) * screenWidth;

// Near (wide) bottom corners — sit at the H-ROI top edge
float hTop = data.hRoiTopFrac * screenHeight;
float blX  = centre - data.halfBotFrac * screenWidth;
float brX  = centre + data.halfBotFrac * screenWidth;

// Far (narrow) top corners — shifted by shear so the box leans with the road
float vTop = data.vRoiTopFrac * screenHeight;
float tlX  = centre + shear - data.halfTopFrac * screenWidth;
float trX  = centre + shear + data.halfTopFrac * screenWidth;

// Connect the 4 corners:
//   bottom-left (blX, hTop) → bottom-right (brX, hTop)
//   → top-right (trX, vTop) → top-left (tlX, vTop) → close
```

Dynamic road band (bends through curves):

```
int n = data.lookAheadPointCount;
if (n >= 2) {
    for (int i = 0; i < n; i++) {
        float x = data.lookAheadColFrac[i] * screenWidth;
        float y = data.lookAheadRowFrac[i] * screenHeight;
        // connect points with a poly-line
        // i = 0 is nearest (bottom), i = n-1 is farthest (top)
    }
    // To draw as a band: trace the left edge (x - halfWidth) forward,
    // then the right edge (x + halfWidth) backward.
    // halfWidth tapers from halfBotFrac (near) to halfTopFrac (far).
}
```

Centroid indicator (live line position):

```
if (data.centroidXFrac >= 0) {
    float cx  = data.centroidXFrac * screenWidth;
    float top = data.vRoiTopFrac   * screenHeight;
    float bot = data.hRoiBotFrac   * screenHeight;
    // vertical line at cx, from top to bot
    // colour: green if data.trackFound, orange if not
}
```

State readout:

```
if (!data.isCalibrated) {
    status.setText("Calibrating " + Math.round(data.calibProgress * 100) + "%");
    // data.calibIssue explains a stall — see the Calibration table
} else if (data.lineHeld) {
    status.setText("Line hidden — holding S=" + data.steering);
} else if (!data.trackFound) {
    status.setText("Track lost");
} else {
    status.setText("Tracking  S=" + data.steering + "  T=" + data.tracking);
}
lineSwatch.setColor(data.lineColorArgb);   // learned colours
floorSwatch.setColor(data.floorColorArgb);
```

Steering debug readout NEW (new v2 fields — optional):

```
debug.setText(String.format("offset %.1f + heading %.1f (%.0f°, c=%.2f)  rejected=%d",
        data.steerOffsetTerm, data.steerHeadingTerm,
        data.headingDeg, data.headingConfidence, data.rejectedBands));
```

Full Example

```
LineFollowerPipeline pipeline = new LineFollowerPipeline();
final int W = 960, H = 1280;    // frame size (RoboPhoneApp: CameraX 1280 × 960, rotated to portrait)

// Optional tuning
LineFollowerPipeline.Config cfg = new LineFollowerPipeline.Config();
cfg.autoColor        = true;    // learn line + floor colours (any two colours)
cfg.cameraTiltDeg    = 45f;     // phone mount angle — measure it on your robot
cfg.bottomIgnoreFrac = 0.20f;   // hide robot body from scan
cfg.trackMemory      = true;    // lock onto line, ignore reflections
cfg.trackingMinValue = 5;       // never send a full stop
pipeline.configure(cfg);

// Register overlay callback — fires automatically on every frame
pipeline.setFrameCallback(data -> {
    myOverlay.drawHRoi(data.hRoiTopFrac, data.hRoiBotFrac);
    myOverlay.drawVRoi(data.vRoiTopFrac, data.steerRefXFrac,
                       data.trackRefXFrac, data.halfBotFrac, data.halfTopFrac);
    myOverlay.drawRoadPath(data.lookAheadColFrac,
                           data.lookAheadRowFrac,
                           data.lookAheadPointCount);
    myOverlay.drawCentroid(data.centroidXFrac, data.trackFound);

    if (data.isCalibrated) {    // drive only once calibration is complete
        robot.setSteeringAngle(data.steering);
        robot.setSpeed(data.tracking);
    }
});

// Phase 1 — Calibration (place robot correctly on the line first: straight, on a straight section)
Yuv f = camera.getYuvFrame();   // packed y / u / v planes
while (!pipeline.calibrateYuv420(f.y, f.u, f.v, W, H)) {
    f = camera.getYuvFrame();
}

// Phase 2 — Normal operation
while (running) {
    f = camera.getYuvFrame();
    pipeline.processFrameYuv420(f.y, f.u, f.v, W, H);
    // all results and overlay data delivered automatically via the callback
}
```

Behaviour Reference

SITUATION	STEERING	TRACKING
Straight line, centred	~0	~100
Line drifting to one side	± small	high
Curve approaching (line leaning)	varies	dropping toward the floor
Approaching a dead end / sharp corner	~0	ramps 100 → 5 with the remaining road
Robot at an angle to a straight line (line crossing the strip at the calibrated column)	≈ ±8 at 15°, ≈ ±17 at 30° *(heading part)*	high

SITUATION	STEERING	TRACKING
Dark object / shadow beside or touching the line	unchanged *(object rejected or trimmed off)*	unchanged
Line hidden for a moment in the middle of the image	last value held for trackHoldFrames	last value held
Hard turn	±50	5–30
Line completely lost	±50	5 *(the floor)*
Back on a straight after a turn	returns to 0	~100
v3: glare / shadow / object over part of the line, line visible before or after it	unchanged *(bridged from memory and continuity)*	unchanged
v3: sharp corner just in front of the robot (below the view)	follows the remembered corner	low
v3: background beyond a table edge at the top of the view	unchanged *(outside the ROI found at calibration)*	unchanged

Recommended Starting Values (lecturer's test program) NEW

For the Blockly test program — **motor steering = (0 − Steering) × c** and **motor power = round(x × (Power / 8) + 5)**, where *Steering* is `result[0]` and *Power* is the Tracking value `result[1]` — the closed-loop simulation (`_investigation/CHANGES_v3.md`) recommends:

SETTING	START WITH	NOTES
Steering gain **c** (Ks)	`1.0` — unchanged	v3 completed all 88 runs with c = 1 in the realistic regime (RMS lateral error 1.1 cm, v2 2.7 cm). `0.5` also works (85 / 88; the 3 failures are the sharp 90° corner with distractors — v2 fails the mirrored corner there) and `steerAutoGain` adapts the scale. No change is needed in the block.
Power mode **x** (Kp)	`0` first, then `1` with `Power / 8` — unchanged	Tracking is now smooth (no stop-and-go), so `x = 1` is safe to try earlier than with v2.
Offset **+ 5**	`5`	If the robot does not move at power 5 (motor dead zone), raise it to 6–8.
Library	defaults	Set `cameraTiltDeg` roughly (it is refined at calibration) and `bottomIgnoreFrac` so the robot body is excluded. If the robot weaves, raise `pursuitLookAhead` (1.5); if it cuts tight curves, lower it (1.0).

These values come from simulation, not from the real robot — verify them on the track (and keep `x = 0` if the measured Blockly loop rate is well below ~15 Hz).

Notes

- The pipeline is **not thread-safe**. Call `calibrate()` / `processFrame()` (and `requestRecalibration()`) from a single thread only — e.g. have a UI button set a flag that the camera thread checks.
- The returned `int[]` is reused on every call. Copy the values if you need them past the next frame.
- Calibration assumes the robot is correctly placed on the line, with floor visible on both sides. If it is not, call `requestRecalibration()` once it is repositioned, then calibrate again. Recalibrate whenever you move to a track with different colours, different lighting or a different tape width.
- A line that jumps sideways by more than the line-lock gate in a single frame (`trackMemoryGateMult` × line width, around the line's predicted position) is treated as leaving the line — this is what makes the lock ignore reflections.
- v3 needs the camera geometry only roughly (`cameraTiltDeg` within ~25°, `verticalFovDeg`): the tilt is learned at calibration and the line width per row is measured. The Android app runs at 960 × 1280 (CameraX picks 1280 × 960, rotated to portrait).
- Very harsh scenes can still defeat colour learning: e.g. a dark floor in deep shadow, with strong texture, where the tape is only ~10 grey-levels darker than the shadowed floor. Even lighting helps.
- Performance (v3): ≈ 0.4 ms (median 0.37–0.41 ms; v2 on the same frames and machine: 2.3–2.4 ms) per frame in YUV colour mode at 960 × 1280 on a desktop CPU (v2: ≈ 0.9 ms) — v3 reads only the pixels along the path, never the whole image; all per-frame buffers are pre-allocated (no garbage per frame, in all three input formats).

RoboPhoneApp — Android Reference Integration

`RoboPhoneApp/` runs the library live on the phone camera (CameraX → packed YUV 4:2:0 → `calibrateYuv420` / `processFrameYuv420`). Everything on screen comes from the `FrameCallback`:

- **Overlay (v3)** CHANGED — the working region (white frame), unreliable cells (red), the traced path coloured by where each point came from (green = measured now, blue = memory, orange = hidden now and filled from memory, yellow = bridged), the look-ahead target (magenta) with a line from the robot, a steering arrow, red ignored robot-body band; with `pathTracking = false` the v2 ⊥. Mapped with the preview's FILL_CENTER crop so it lines up with the camera image.
- **HUD** — STEER bar (−50…+50), TRACK bar (min…100 %), status (calibration progress and problems / TRACKING / TRACK LOST), FPS, learned LINE and FLOOR colour swatches, colour separation, and in v3 the path confidence and the share of the path from memory.
- **RECALIBRATE** button — re-learns the colours and the straight-ahead reference (place the robot on the line first).

The app builds the library **from source** through a Gradle composite build (`includeBuild("../LineFollowerLibrary")`), so library changes show up in the app on the next build — no jar copying. Library tests: `cd LineFollowerLibrary && gradlew test` (format / colour unit tests plus camera-simulator scenes: shadows, wood grain, turns, line loss, distractor tapes, dead ends).

Unit tests: all **45 tests pass** with the v3 defaults — the 26 earlier tests (expectations adapted where the approach changed; the v1/v2 tests run with `pathTracking = false`) and 19 v3 scenario tests (`PathTrackingTest`: straight, curves both ways, S-curve, line leaving the side, start at an angle / offset, table edge, blob and bright stripe beside the line, the robot's own shadow, glare on a straight and on a curve, a 4-colour-pair × 6-disturbance matrix, disturbances during calibration, the new `FrameData` fields, option switches). `gradlew jar` and the app build do not run the tests.